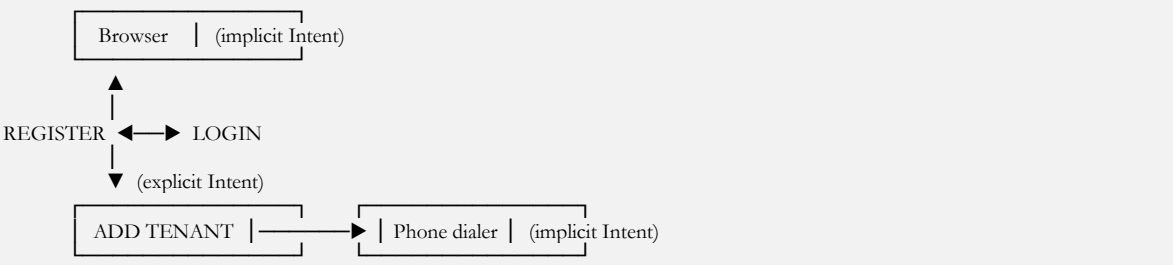
Strathmore
UNIVERSITY

SCHOOL OF COMPUTING AND ENGINEERING SCIENCES
BACHELOR IN BUSINESS INFORMATION TECHNOLOGY
BBT 3.2: Mobile Application Development
Language: Kotlin
Lesson 6
Navigation Practical 1 — Moving Between Screens with Intents
App Navigation, Part 1 of 3

What we are building

In the last practical, you built the **Add Tenant** screen. In this practical, you add a Login and a Register screen in front of it, move between them with **explicit Intents**, send data from one screen to another, and use **implicit Intents** to ask other apps (the browser and the phone dialer) to do things for us.



We will work through:

1. Create the Login screen as a new Activity
2. Open another Activity with an explicit Intent
3. Make Login the first screen of the app
4. Create the Register screen
5. Send data between Activities with putExtra
6. Open a website with an implicit Intent
7. Open the phone dialer with an implicit Intent

This practical continues from the last one. You need your project with View Binding and Data Binding enabled, Tenant.kt with the summary() function, and the Add Tenant screen in activity_main.xml.

Throughout this guide, boxes marked **Question** answer the things students most often get stuck on. If something goes wrong, check the nearest Question box first, then the **Common errors** table at the end.

Everywhere this guide shows com.example.tenantmanagementsystemgroupa, use your own package name. You can find it on the first line of MainActivity.kt.

NOTE: Please read the code and try understanding do not just copy and paste

---

Activities and Intents

So far, our app has one screen: **MainActivity**. Each full screen in an Android app can be its own **Activity**. To move from one Activity to another, we send an **Intent**: a message that says "please do this".

Type	What it says	Example in our app
Explicit Intent	"Open this exact screen"	Login → MainActivity
Implicit Intent	"I want this done; any app that can do it may respond"	Open a website, dial a number

---

PART A — Create the Login screen

Step 1 — Create LoginActivity

On the left, right-click:

```
com.example.tenantmanagementsystemgroupa
```

Then choose:

```
New → Activity → Empty Views Activity
```

A window appears. Fill it in:

```
Activity Name:     LoginActivity
Layout Name:       activity_login
Launcher Activity: leave UNTICKED
```

Click **Finish**.

Android Studio creates two files for you:

```
kotlin+java
└── com.example.tenantmanagementsystemgroupa
    └── LoginActivity.kt


res
└── layout
    └── activity_login.xml
```

It also registers the Activity in AndroidManifest.xml for you.

> **Question: Why use "New → Activity" and not "New → Kotlin Class/File" like before?**
> Every Activity must be listed in AndroidManifest.xml, or the app crashes with ActivityNotFoundException when you try to open it. The Activity wizard adds that entry automatically.
>
> **Question: I don't see "Empty Views Activity".**
> Look for it under **New → Activity → Gallery...** and choose **Empty Views Activity**. Do not choose "Empty Activity" (that one uses Jetpack Compose, not XML layouts).

Step 2 — Design activity_login.xml

Open res → layout → activity_login.xml, click **Code**, delete everything and paste:

```
<?xml version="1.0" encoding="utf-8"?>


<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true"
```

```
    android:padding="24dp">

    <TextView
        android:id="@+id/loginTitleTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="TENANT MANAGEMENT SYSTEM"
        android:textSize="22sp"
        android:textStyle="bold"
        android:layout_marginTop="48dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <TextView
        android:id="@+id/loginSubtitleTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Log in to continue"
        android:textSize="16sp"
        android:layout_marginTop="8dp"
        app:layout_constraintTop_toBottomOf="@id/loginTitleTextView"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <EditText
        android:id="@+id/emailEditText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Email"
        android:inputType="textEmailAddress"
        android:minHeight="48dp"
        android:layout_marginTop="32dp"
        app:layout_constraintTop_toBottomOf="@id/loginSubtitleTextView"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <EditText
        android:id="@+id/passwordEditText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword"
        android:minHeight="48dp"
        android:layout_marginTop="16dp"
        app:layout_constraintTop_toBottomOf="@id/emailEditText"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <Button
        android:id="@+id/loginButton"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="LOGIN"
        android:layout_marginTop="24dp"
        app:layout_constraintTop_toBottomOf="@id/passwordEditText"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <TextView
        android:id="@+id/registerTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Don't have an account? Register"
        android:textColor="#1F4E79"
        android:padding="12dp"
        android:layout_marginTop="16dp"
        app:layout_constraintTop_toBottomOf="@id/loginButton"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

> **Question: What does `android:fitsSystemWindows="true"` do?**
> On newer phones, apps draw behind the status bar at the top of the screen. This line adds space so your content isn't hidden behind the clock and battery icons.
>
> **Question: Why does the root have `padding="24dp"` and the views have no start or end margins?**
> The padding keeps every view 24dp away from the screen edges, so we don't need to repeat the margin on each view.
>
> **Question: Why `inputType="textPassword"`?**
> It hides the characters with dots as the user types.

Step 3 — Write LoginActivity.kt

Open LoginActivity.kt. The wizard added some code, including `enableEdgeToEdge()` and a `ViewCompat...` block. Delete everything **except the package line**, then paste:

```
import android.content.Intent
import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.example.tenantmanagementsystemgroupa.databinding.ActivityLoginBinding

class LoginActivity : AppCompatActivity() {

    private lateinit var binding: ActivityLoginBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        binding = ActivityLoginBinding.inflate(layoutInflater)

        setContentView(binding.root)

        binding.loginButton.setOnClickListener {

            val email = binding.emailEditText.text.toString().trim()

            val password = binding.passwordEditText.text.toString()

            if (email.isEmpty() || password.isEmpty()) {
                Toast.makeText(this, "Please enter your email and password", Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }

            val intent = Intent(this, MainActivity::class.java)

            startActivity(intent)

            finish()
        }
    }
}
```

This is an **explicit Intent**: we name the exact screen we want to open, `MainActivity`.

```
Intent(this, MainActivity::class.java)
       |              |
    from here      go to here
```

The next part is for people experiencing error after creating login activity if you do not have an error go straight to step 4.

> **Note: ActivityLoginBinding is red?**
> At this point, ActivityLoginBinding (and binding.loginButton, binding.emailEditText and so on) may show as red errors. This happens because Android Studio hasn't generated the binding class for your new layout yet. To fix it:
>
> 1. Open the main menu (☰) at the top-left of Android Studio.
> 2. Click **File → Invalidate Caches...**
> 3. Tick all the boxes.
> 4. Click **Invalidate and Restart**.

> **Question: What is a Toast?**
> A small message that pops up at the bottom of the screen for a moment and then disappears.
>
> **Question: What does finish() do?**
> It closes LoginActivity after opening MainActivity. Without it, pressing Back on the Dashboard would return to the Login screen, which is not what users expect after logging in.
>
> **Question: Any email and password logs me in. Is that right?**
> Yes, for now. We have no database of users yet, so we only check that the fields are not empty. Real accounts come when we cover data storage.

Step 4 — Make LoginActivity the first screen

When the app opens, Android shows the Activity marked as the **launcher**. At the moment, that is MainActivity. We change it to LoginActivity.

On the left, open:

```
app
└── manifests
    └── AndroidManifest.xml
```

Find the <activity> entries. You will see something similar to:

```
    <activity
  android:name=".MainActivity"
  android:exported="true"
  android:windowSoftInputMode="adjustResize">
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />

    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>
<activity
  android:name=".LoginActivity"
  android:exported="false"
  android:windowSoftInputMode="adjustResize" />
```

Change them to:

```
    <activity
  android:name=".LoginActivity"
  android:exported="true"
  android:windowSoftInputMode="adjustResize">
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />


    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>


<activity
  android:name=".MainActivity"
  android:exported="false"
  android:windowSoftInputMode="adjustResize" />
```

In short: the whole <intent-filter> block moves from MainActivity to LoginActivity, and the exported values swap.

> **Question: What does the <intent-filter> with MAIN and LAUNCHER mean?**
> It tells Android "this is the screen to open when the user taps the app icon". Only one Activity should have it.
>
> **Question: What does exported mean?**
> Whether other apps (like the phone's home screen) are allowed to open this Activity. The launcher Activity must be true. Screens that only our own app opens can be false.
>
> **Question: My <activity> entries have other lines, like android:label or android:theme.**
> Keep them. Only move the <intent-filter> and change exported.

Checkpoint. Run the app. The Login screen appears first. Enter any email and password and tap **LOGIN**. You should see the Add Tenant screen from the last practical (MainActivity). Press Back: the app closes instead of returning to Login, because of finish().

PART B — Create the Register screen and send data between Activities

Step 5 — Create RegisterActivity

Right-click com.example.tenantmanagementsystemgroupa and choose **New → Activity → Empty Views Activity**:

```
Activity Name:    RegisterActivity
Layout Name:      activity_register
Launcher Activity: leave UNTICKED
```

Click **Finish**.

Step 6 — Design activity_register.xml

Open activity_register.xml, click **Code**, delete everything and paste:

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true"
    android:padding="24dp">

    <TextView
        android:id="@+id/registerTitleTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Create an account"
        android:textSize="22sp"
        android:textStyle="bold"
        android:layout_marginTop="48dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <EditText
        android:id="@+id/fullNameEditText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Full name"
        android:inputType="textPersonName"
        android:minHeight="48dp"
        android:layout_marginTop="32dp"
        app:layout_constraintTop_toBottomOf="@id/registerTitleTextView"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <EditText
        android:id="@+id/registerEmailEditText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Email"
        android:inputType="textEmailAddress"
        android:minHeight="48dp"
        android:layout_marginTop="16dp"
        app:layout_constraintTop_toBottomOf="@id/fullNameEditText"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <EditText
        android:id="@+id/registerPasswordEditText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword"
        android:minHeight="48dp"
```

```
        android:layout_marginTop="16dp"
        app:layout_constraintTop_toBottomOf="@id/registerEmailEditText"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <Button
        android:id="@+id/registerButton"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="REGISTER"
        android:layout_marginTop="24dp"
        app:layout_constraintTop_toBottomOf="@id/registerPasswordEditText"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <TextView
        android:id="@+id/loginLinkTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Already have an account? Log in"
        android:textColor="#1F4E79"
        android:padding="12dp"
        android:layout_marginTop="16dp"
        app:layout_constraintTop_toBottomOf="@id/registerButton"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Step 7 — Write RegisterActivity.kt

Open RegisterActivity.kt. Delete everything **except the package line**, then paste:

```
import android.content.Intent
import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.example.tenantmanagementsystemgroupa.databinding.ActivityRegisterBinding

class RegisterActivity : AppCompatActivity() {

    private lateinit var binding: ActivityRegisterBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        binding = ActivityRegisterBinding.inflate(layoutInflater)

        setContentView(binding.root)

        binding.registerButton.setOnClickListener {

            val fullName = binding.fullNameEditText.text.toString().trim()

            val email = binding.registerEmailEditText.text.toString().trim()

            val password = binding.registerPasswordEditText.text.toString()

            if (fullName.isEmpty() || email.isEmpty() || password.isEmpty()) {
                Toast.makeText(this, "Please fill in all fields", Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }

            Toast.makeText(this, "Account created. Please log in.", Toast.LENGTH_SHORT).show()

            val intent = Intent(this, LoginActivity::class.java)

            intent.putExtra("EMAIL", email)

            intent.flags = Intent.FLAG_ACTIVITY_CLEAR_TOP
```

```
        startActivity(intent)
    }

    binding.loginLinkTextView.setOnClickListener {
        finish()
    }
  }
}
```

NOTE: This time the Intent carries **data**. putExtra attaches the email to the Intent, like writing a note on an envelope before sending it:



Question: What is "EMAIL"?

A key: the label for the data. The screen that receives the Intent uses the same key to read the value. It must be spelled exactly the same on both sides.

Question: What does FLAG_ACTIVITY_CLEAR_TOP do?

The Login screen is already open underneath Register. Without this flag, Android would open a second Login screen on top, and pressing Back would show Register again. With it, Android closes Register and the old Login, and opens a fresh Login that receives our email.

Question: Why does the "Log in" link only call finish()?

Register was opened from Login, so Login is still underneath. Closing Register reveals it again. We don't need a new Intent.

Step 8 — Open Register from Login, and read the email

Open LoginActivity.kt.

First, underneath setContentView(binding.root), add:

```
    val registeredEmail = intent.getStringExtra("EMAIL")

    if (registeredEmail != null) {
        binding.emailEditText.setText(registeredEmail)
    }
```

Then, at the end of onCreate() (after the loginButton block, before the final } of onCreate), add:

```
    binding.registerTextView.setOnClickListener {

        val intent = Intent(this, RegisterActivity::class.java)

        startActivity(intent)
    }
```

Your complete LoginActivity.kt:

```
package com.example.tenantmanagementsystemgroupa

import android.content.Intent
import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.example.tenantmanagementsystemgroupa.databinding.ActivityLoginBinding

class LoginActivity : AppCompatActivity() {

    private lateinit var binding: ActivityLoginBinding
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        binding = ActivityLoginBinding.inflate(layoutInflater)

        setContentView(binding.root)

        val registeredEmail = intent.getStringExtra("EMAIL")

        if (registeredEmail != null) {
            binding.emailEditText.setText(registeredEmail)
        }

        binding.loginButton.setOnClickListener {

            val email = binding.emailEditText.text.toString().trim()

            val password = binding.passwordEditText.text.toString()

            if (email.isEmpty() || password.isEmpty()) {
                Toast.makeText(this, "Please enter your email and password", Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }

            val intent = Intent(this, MainActivity::class.java)

            startActivity(intent)

            finish()
        }

        binding.registerTextView.setOnClickListener {

            val intent = Intent(this, RegisterActivity::class.java)

            startActivity(intent)
        }
    }
}
```

> **Question: Where does `intent` (lowercase) come from in `intent.getStringExtra`?**
> Every Activity automatically has an `intent` property: the Intent that opened it. When Register opens Login, this is the Intent with the email attached.
>
> **Question: Why check `if (registeredEmail != null)`?**
> When the app opens normally from the icon, nobody attached an email, so `getStringExtra("EMAIL")` gives `null`. We only fill in the field when an email was actually sent.

Checkpoint. Run the app.

1. On Login, tap **Don't have an account? Register**.
2. Fill in the three fields and tap **REGISTER**.
3. You return to Login, and the email field is already filled in.
4. Enter any password and tap **LOGIN**.

PART C — Implicit Intents

An explicit Intent names the exact screen to open. An **implicit Intent** only describes what you want done, and Android finds an app on the phone that can do it.

```
Explicit:   "Open MainActivity."
            → Android opens our MainActivity.

Implicit:   "Show this web page."
            → Android asks: which apps can show web pages?
            → Chrome (or another browser) opens it.
```

An implicit Intent has two main parts:

Part	Meaning	Examples
Action	What you want to do	ACTION_VIEW (show something), ACTION_DIAL (dial a number), ACTION_SEND (share something)
Data	What to do it with	A web address, a phone number, some text

Step 9 — Add a help link to the Login screen

Open activity_login.xml (Code view). Just **before** the last line, </androidx.constraintlayout.widget.ConstraintLayout>, paste:

```
<TextView
    android:id="@+id/helpTextView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Need help? Visit our website"
    android:textColor="#1F4E79"
    android:padding="12dp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

This places the link at the bottom of the screen, centred.

Step 10 — Open the website with ACTION_VIEW

Open LoginActivity.kt. Add this import at the top, with the other imports:

```
import android.net.Uri
```

At the end of onCreate() (after the registerTextView block, before the final } of onCreate), add:

```
    binding.helpTextView.setOnClickListener {

        val intent = Intent(Intent.ACTION_VIEW, Uri.parse("https://www.strathmore.edu"))

        startActivity(intent)
    }
```

NOTE: if you have an error hoover around Uri and press alt +Enter then import Uri.

Compare it with the explicit Intent in the same file:

```
Intent(this, MainActivity::class.java)
     |        |
   from     go to this exact screen

Intent(Intent.ACTION_VIEW, Uri.parse("https://www.strathmore.edu"))
            |                  |
       what to do         what to do it with
```

> **Question: What is Uri.parse(...)?**
> It turns a piece of text into a **Uri**, a standard address that Android understands. The start of the address (https:) tells Android what kind of data it is, which helps it find the right app.

Question: Which browser opens?

Whichever the user has set as their default. If there is more than one and no default, Android shows a chooser. Our code doesn't know or care which one; that's the point of an implicit Intent.

Question: The app crashes with ActivityNotFoundException when I tap the link.

No app on the device can handle the Intent. This can happen on emulators without a browser. Use an emulator image that includes Google Play or Chrome, or test on a real phone.

Checkpoint. Run the app and tap **Need help? Visit our website** on the Login screen. The browser opens the Strathmore website. Press Back to return to your app.

Step 11 — Add a CALL TENANT button to the Add Tenant screen

Open activity_main.xml (Code view). This is the Data Binding layout from the last practical.

Just **before** the line </androidx.constraintlayout.widget.ConstraintLayout> (near the end, above </layout>), paste:

```
    <Button
        android:id="@+id/callButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="CALL TENANT"
        android:layout_marginStart="24dp"
        app:layout_constraintTop_toBottomOf="@id/tenantResultTextView"
        app:layout_constraintStart_toStartOf="parent" />
```

Question: My file ends with </layout>. Where exactly does the button go?

Inside the ConstraintLayout, so above </androidx.constraintlayout.widget.ConstraintLayout>, which is itself above </layout>. The button must be inside the ConstraintLayout for its constraints to work.

Step 12 — Dial the saved tenant with ACTION_DIAL

Open MainActivity.kt. Replace everything **except the package line** with:

```
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.example.tenantmanagementsystemgroupa.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    private var lastTenant: Tenant? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        binding = ActivityMainBinding.inflate(layoutInflater)

        setContentView(binding.root)

        binding.saveButton.setOnClickListener {

            val name = binding.tenantNameEditText.text.toString()

            val phone = binding.phoneEditText.text.toString()

            val rent = binding.rentEditText.text.toString()

            val tenant = Tenant(name, phone, rent)

            binding.tenant = tenant
```

```
            lastTenant = tenant
        }

        binding.callButton.setOnClickListener {

            val tenant = lastTenant

            if (tenant == null) {
                Toast.makeText(this, "Save a tenant first", Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }

            val intent = Intent(Intent.ACTION_DIAL, Uri.parse("tel:${tenant.phone}"))

            startActivity(intent)
        }
    }
}
```

The SAVE code is the same as the last practical, with one new line: `lastTenant = tenant` remembers the most recently saved tenant so the CALL button knows whose number to dial.

> **Question: Does this make a phone call?**
> No. `ACTION_DIAL` opens the phone's dialer with the number already typed in. The user still has to press the call button. That's why it doesn't need any special permission.
>
> **Question: What is `"tel:"`?**
> It tells Android what kind of data this is: a telephone number. `"tel:0712345678"` is a phone number the same way `"https://..."` is a web address.
>
> **Question: What does `${tenant.phone}` mean?**
> A string template, like `$name` before. The curly brackets are needed because we're reading a property (`.phone`), not just a variable name.
>
> **Question: Why `val tenant = lastTenant` instead of using `lastTenant` directly?**
> `lastTenant` is a `var`, so Kotlin worries it could change between the null check and the next line. Copying it into a `val` first lets Kotlin be sure it's not null after the check.
>
> **Question: Why `private var lastTenant: Tenant? = null`?**
> The `?` means it may be empty. Before the first SAVE, there is no tenant, so it starts as `null`.

Checkpoint. Run the app and log in.

1. Tap **CALL TENANT** before saving: a message says "Save a tenant first".
2. Enter a tenant and tap **SAVE**.
3. Tap **CALL TENANT**: the dialer opens with the tenant's phone number filled in.

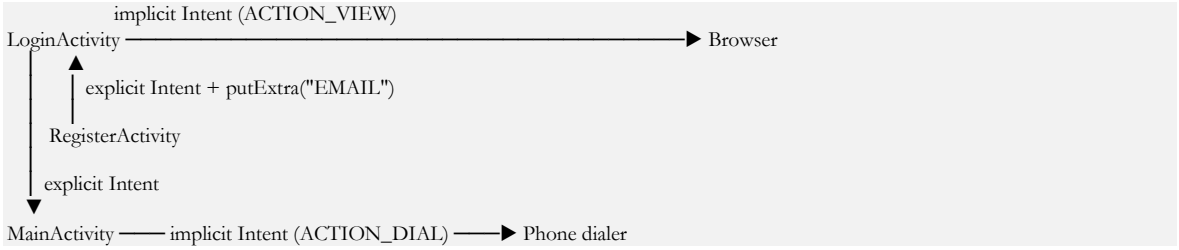
Run the complete app

1. The app opens on **Login**. Tap **Need help?** and the website opens. Press Back.
2. Tap **Register**, fill in the form and tap **REGISTER**. You return to Login with the email filled in.
3. Enter a password and tap **LOGIN**. The Add Tenant screen opens.
4. Save a tenant and tap **CALL TENANT**. The dialer opens.
5. Return to the app and press Back. The app closes rather than returning to Login.

Final project structure

```
TenantManagementSystem
└── app
    ├── manifests
    │   └── AndroidManifest.xml
    ├── kotlin+java
    │   └── com.example.tenantmanagementsystemgroupa
    │       ├── LoginActivity.kt
    │       ├── RegisterActivity.kt
    │       ├── MainActivity.kt
    │       └── Tenant.kt
    └── res
        └── layout
            ├── activity_login.xml
            ├── activity_register.xml
            └── activity_main.xml
```

How the screens connect:



Common errors

What you see	Cause	Fix
ActivityNotFoundException when opening Login or Register	Activity not in the manifest	Create Activities with **New → Activity**, or add the <activity> entry
App still opens on the Add Tenant screen	Launcher <intent-filter> still on MainActivity	Move it to LoginActivity (Step 4)
Two app icons on the emulator	Both Activities have the launcher <intent-filter>	Keep it only on LoginActivity
Crash when opening Login or Register	Leftover ViewCompat...R.id.main code	Replace the whole file (Steps 3 and 7)
Email not filled in after Register	Key mismatch in putExtra/getStringExtra	Use "EMAIL" exactly on both sides
Register opens a second Login, and Back shows Register again	Missing FLAG_ACTIVITY_CLEAR_TOP	Add the flag (Step 7)
Uri is red	Missing import	Add import android.net.Uri
ActivityNotFoundException when opening the website	No browser on the emulator	Use an emulator image with Google Play, or a real phone

What you see	Cause	Fix
callButton is red in MainActivity	Button not added, or added outside the ConstraintLayout	Check Step 11, then **Build → Rebuild Project**
CALL TENANT always says "Save a tenant first"	lastTenant = tenant missing from the SAVE code	Add it (Step 12)

Try it yourself

1. **Share a tenant.** Add a SHARE button to the Add Tenant screen that shares the saved tenant's details with another app (WhatsApp, Messages, Gmail):

```
val intent = Intent(Intent.ACTION_SEND)
intent.type = "text/plain"
intent.putExtra(Intent.EXTRA_TEXT, tenant.summary())
startActivity(Intent.createChooser(intent, "Share tenant"))
```

1. **Welcome the user.** Send the email from LoginActivity to MainActivity with putExtra, read it in MainActivity with getStringExtra, and show it in a Toast: "Logged in as ...".
2. **Validate the tenant.** Before saving, check that none of the three fields is empty. Show an error on the empty field with .error = "Required".

What you learned today

```
STEP 1   Multiple Activities
       "My app can have more than one screen."
            ↓
STEP 2   Explicit Intents
       "I can open a specific screen in my app."
            ↓
STEP 3   putExtra and getStringExtra
       "I can send data from one screen to another."
            ↓
STEP 4   Implicit Intents
       "I can ask other apps to do things for me:
        open a website, dial a number."
```

Coming up next

In **Practical 2**, the Add Tenant screen becomes one of several screens that live inside MainActivity: a Dashboard, Add Tenant and View Tenants. You'll build them as **Fragments** and move between them with the **Navigation Component**.